

UNIVERSITY OF USTHB
Department of Computer Science

Mini-Project Report

Medical Clinic Management System

Object-Oriented Programming (OOP) — Java

Prepared by

Bounoua Akram
Kadouche Mohamed Chakib
Boumara brahim
Ouznaji Sohaib
Bouzouad Myriam Malak

Academic Year: ACAD L2 2025 - 2026

2. Introduction

This report documents the design and implementation of a Java console application developed as part of the **Object-Oriented Programming** module . The application simulates the daily management of a medical clinic and serves as a practical demonstration of the core **OOP** principles taught throughout the course.

2.1 Context

Medical institutions increasingly rely on software systems to manage their day-to-day operations. The goal of this project is to model a simplified version of such a system, covering the main actors and processes of a clinic: patients, medical staff, consultations, and prescriptions.

Because of our limited knowledge in **GUI** (**G**raphical **U**ser **I**nterface) java we tried to create some sort of a **CLI** (**C**ommand **L**ine **I**nterface) with some pixel art to get a decent User Experience .

2.2 Objectives

The application is designed to achieve the following objectives:

- Allow clinic staff to **register** and **manage** patient records
- Maintain a roster of doctors and nurses with their professional details
- Schedule consultations between patients and doctors, with availability checking
- Issue and manage medical prescriptions linked to consultations
- Search consultation history by patient or by doctor
- Display a **live dashboard** summarising clinic activity and revenue

2.3 Scope

The application operates entirely in console mode with an interactive text menu. It does not use a database; all data lives in memory during a session. On startup, a set of demo data is loaded automatically so that all features can be tested immediately without manual data entry.

3. OOP Implementation

This section presents each OOP concept implemented in the project, accompanied by representative code extracts.

3.1 Encapsulation

All class attributes are declared **private**. Access to these attributes is provided exclusively through **getter** and **setter** methods. Setters perform business-rule validation before accepting a new value. This protects the integrity of sensitive medical data and prevents illegal states within the application.

Example — the setter for **groupeSanguin** in the **Patient** class rejects any value that is not a recognised blood type:

```
// modele/Patient.java
private static final List<String> GROUPES_VALIDES =
    List.of("A+", "A-", "B+", "B-", "O+", "O-", "AB+", "AB-");

public void setGroupeSanguin(String groupe) {
    if (GROUPES_VALIDES.contains(groupe)) {
        this.groupeSanguin = groupe;
    } else {
        System.out.println("Invalid blood type: " + groupe);
        this.groupeSanguin = "Unknown";
    }
}
```

The same pattern is applied to **salaire** (must be ≥ 0) in **PersonnelMedical** and **tarifConsultation** (must be > 0) in **Medecin**.

```
Groupe sanguin (ex: A+, O-) : k-
A Groupe sanguin invalide : 'k-'. Valeurs acceptées : [A+, A-, B+, B-, O+, O-, AB+, AB-]
```

Figure 3.1 — Console output showing validation rejection of an invalid blood type

3.2 Inheritance

The class hierarchy is built using the **extends** keyword. Each subclass calls **super()** in its constructor to initialise the attributes inherited from its parent, avoiding duplication and enforcing a clean chain of initialisation.

The full hierarchy is:

- Personne -> Patient
- Personne -> PersonnelMedical -> Medecin
- Personne -> PersonnelMedical -> Infirmier

Example — the Medecin constructor calling super() up through PersonnelMedical to Personne:

```
// modele/Medecin.java
public Medecin(String identifiant, String nom, String prenom,
                String dateNaissance, String telephone,
                String matricule, String dateEmbauche, double salaire,
                String specialite, String numeroOrdre, double
tarifConsultation) {

    // Calls PersonnelMedical constructor, which calls Personne constructor
    super(identifiant, nom, prenom, dateNaissance, telephone,
          matricule, dateEmbauche, salaire);

    this.specialite = specialite;
    this.numeroOrdre = numeroOrdre;
    setTarifConsultation(tarifConsultation);
}
```

3.3 Polymorphism

The method `afficherProfil()` is declared abstract in the parent class `Personne` and overridden with `@Override` in each concrete subclass. This allows the application to store objects of different types in a single `List<Personne>` and call `afficherProfil()` on each, with Java automatically dispatching to the correct implementation at runtime.

Example — polymorphic display of all staff members in `CliniqueMedicale`:

```
// gestion/CliniqueMedicale.java
public void afficherToutLePersonnel() {
    List<Personne> toutLePersonnel = new ArrayList<>();
    toutLePersonnel.addAll(medecins);
    toutLePersonnel.addAll(infirmiers);

    for (Personne p : toutLePersonnel) {
        // Java calls the correct afficherProfil() at runtime
        // depending on whether p is a Medecin or an Infirmier
        p.afficherProfil();
    }
}
```

Constructor overloading is also used in the **Consultation** class, which provides two constructors: one that accepts clinical notes and one that does not.

```
// modele/Consultation.java

// Constructor WITH clinical notes
public Consultation(Patient patient, Medecin medecin, String date,
                    String diagnostic, String notesCliniques) { ... }

// Constructor WITHOUT clinical notes (overload)
public Consultation(Patient patient, Medecin medecin,
                    String date, String diagnostic) {
    this(patient, medecin, date, diagnostic, "No clinical notes");
}
```

3.4 Abstraction

The class **Personne** is declared abstract, meaning it cannot be instantiated directly. It defines **afficherProfil()** as an abstract method, which acts as a contract that every subclass must fulfil. This guarantees that any object stored as a **Personne** reference will always be able to display its profile, regardless of its concrete type.

```
// modele/Personne.java
public abstract class Personne {
    private String identifiant;
    private String nom;
    private String prenom;
    // ...
    // Abstract method -- every subclass must provide its own
    implementation
    public abstract void afficherProfil();

    // Concrete shared method available to all subclasses
    public int calculerAge() {
        // calculates age from dateNaissance
    }
}
```

PersonnelMedical is also abstract for the same reason: there is no meaningful notion of a generic medical staff member independent of their specific role.

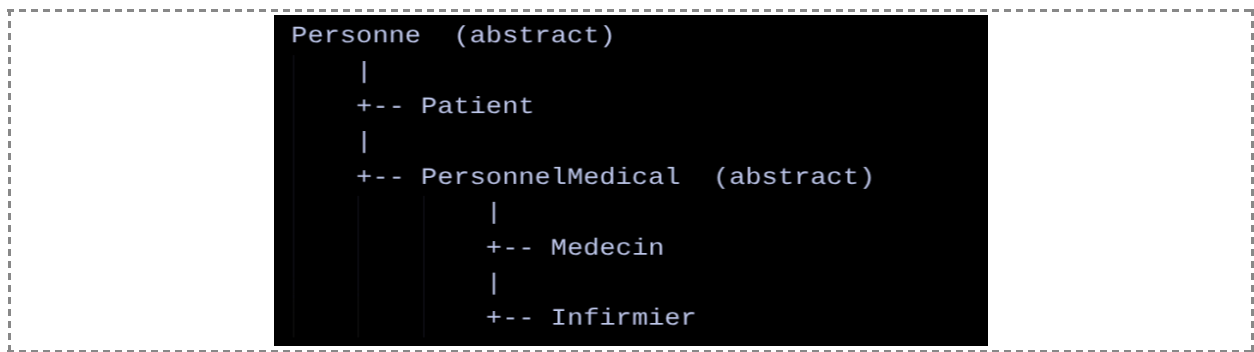


Figure 3.4 — Class hierarchy diagram showing abstract classes

3.5 Exception Handling

Four custom exception classes are defined in the **exception package**, each targeting a specific business error scenario. They all extend **Exceptions** , making them checked exceptions that must be explicitly handled at the call site.

Exception	Thrown when
PatientInexistentException	No patient found for the given ID
MedecinIndisponibleException	Doctor already has a booking at that time slot
DossierMedicalException	Medical file is invalid or inaccessible
OrdonnanceInvalideException	Prescription has no medications or no diagnosis

The **try-catch-finally** pattern is used in the consultation scheduling and prescription creation methods. The finally block runs regardless of whether an exception occurred, and is used to log the end of each operation attempt.

```
// gestion/CliniqueMedicale.java
public Consultation creerConsultation(...)
    throws MedecinIndisponibleException, PatientInexistantException {

    try {
        Patient patient = rechercherPatient(idPatient);
        // may throw PatientInexistantException
        verifierDisponibiliteMedecin(medecin, date);    // may throw
        MedecinIndisponibleException
        // ... create and register the consultation

    } catch (PatientInexistantException | MedecinIndisponibleException e) {
        throw e; // propagate to the menu for user-facing display
    } finally {
        // Always executes -- Logs end of operation
        System.out.println("[End of consultation creation attempt]");
    }
}
```

```
- Nouvelle Consultation -
ID du patient   : P003
ID du médecin  : M001
Date (YYYY-MM-DD HH:MM) : 2026-05-03 10:30
Diagnostic      : malade
Notes cliniques (Entrée pour passer) : ce malade est malade
[Fin de la tentative de création de consultation]
X Dr. Karim Bouzid est indisponible le 2026-05-03 10:30 - un rendez-vous existe déjà à ce créneau.
```

Figure 3.5 — Console output when a doctor is unavailable at the requested time

4. Features

The application presents an interactive console menu divided into six functional areas. Demo data is loaded on startup so that every feature can be tested immediately without manual input.

4.1 Patient Management

The patient management menu allows the user to register new patients, look up and display a patient's full medical file (personal details, blood type, and medical history), list all registered patients in a structured tree format, and add new medical history entries to an existing patient record.

```
== TOUS LES PATIENTS (3) ==

P001 - Amira Benali
├── Age           : 31 ans
├── Date naissance : 1995-03-14
├── Téléphone     : 0555123456
├── Groupe sanguin : A+
└── Antécédents   : 2 enregistré(s)

P002 - Yacine Khelifi
├── Age           : 37 ans
├── Date naissance : 1988-07-22
├── Téléphone     : 0661987654
├── Groupe sanguin : O-
└── Antécédents   : 1 enregistré(s)
```

Figure 4.1 — Listing all registered patients

```
ID du patient : P003

┌────────────────────────────────────────────────────────────────────────────────┐
│ DOSSIER MÉDICAL DU PATIENT                                                    │
└────────────────────────────────────────────────────────────────────────────────┘
👤 PATIENT : Sara Zidane
ID       : P003
Âge      : 24 ans
Téléphone : 0770456789
Groupe   : B+
N° Sécurité Sociale : 2011105059003
Groupe Sanguin : B+
Antécédents médicaux :
    Aucun antécédent enregistré.
```

Figure 4.2 — Displaying a patient's full medical file

4.2 Medical Staff Management

Doctors and nurses can be added through dedicated forms. All staff can be displayed together using the polymorphic display method, which calls the correct `afficherProfil()` implementation for each type automatically.

4.3 Consultation Scheduling

When creating a consultation, the system first verifies that the chosen doctor does not already have a booking at the requested date and time. If the slot is taken, a `MedecinIndisponibleException` is raised and the user is informed. Once validated, the consultation is created with either a full or simplified constructor depending on whether clinical notes are provided.

== TOUTES LES CONSULTATIONS (3) ==	
C001 - 2026-05-03 09:00 Statut: Terminée	
Patient	: Amira Benali (ID: P001)
Médecin	: Dr. Karim Bouzid - Cardiologie
Diagnostic	: Insuffisance cardiaque légère
Notes	: Patient essoufflé à l'effort. ECG recommandé.
C002 - 2026-05-03 10:30 Statut: Planifiée	
Patient	: Yacine Khelifi (ID: P002)
Médecin	: Dr. Nadia Ferhat - Pédiatrie
Diagnostic	: Rhinopharyngite aiguë
Notes	: Aucune note clinique

Figure 4.4 — List Consultations

▲ Erreur lors de la création : Médecin introuvable : M005

Figure 4.5 — Error output when the doctor is unavailable

4.4 Prescription Management

Prescriptions are linked to an existing consultation and require a non-empty diagnosis to be created. Medications can be added one by one with their dosage, duration, and contraindications. A validation step confirms that the prescription contains at least one medication before it is finalised and displayed.

ID de l'ordonnance : ORD001	
✓ Ordonnance ORD001 validée.	
ORDONNANCE MÉDICALE	
N° Ordonnance	: ORD001
Date émission	: 2026-05-03
Patient	: Amira Benali
Médecin	: Dr. Karim Bouzid (Cardiologie)
Diagnostic	: Insuffisance cardiaque légère
Médicaments prescrits (2) :	
 Ramipril	
Dosage:	5mg - 1 fois/jour
Durée:	3 mois
Contre-indications:	Insuffisance rénale sévère
 Furosémide	
Dosage:	40mg - matin
Durée:	1 mois
Contre-indications:	Aucune contre-indication connue

Figure 4.6 — Adding a medication to a prescription

4.5 Search

The user can search for all consultations associated with a specific patient or a specific doctor by entering their ID. Results are displayed in the same tree format used throughout the application.

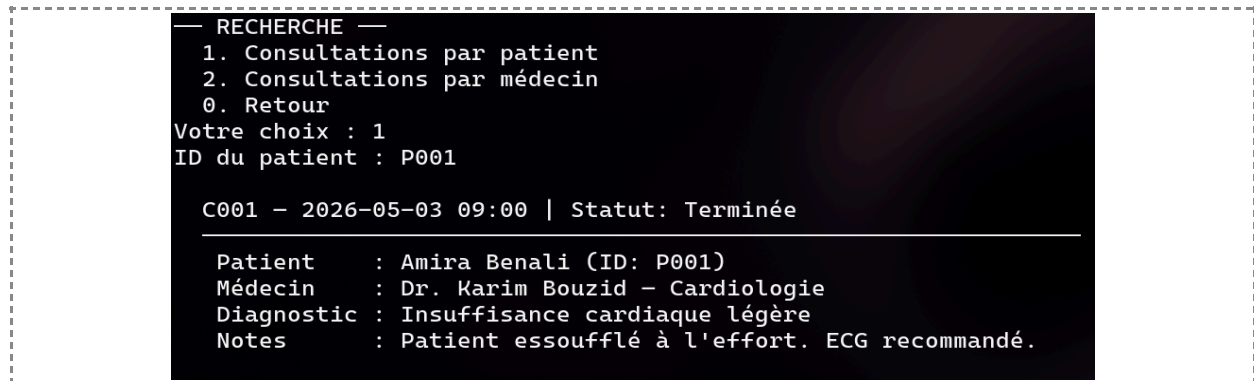


Figure 4.8 — Search results for consultations by patient

4.6 Dashboard

The dashboard provides a summary of the clinic's current state: total number of patients, doctors, nurses, consultations (including a count for today), prescriptions issued, and the total revenue calculated from all non-cancelled consultation fees.

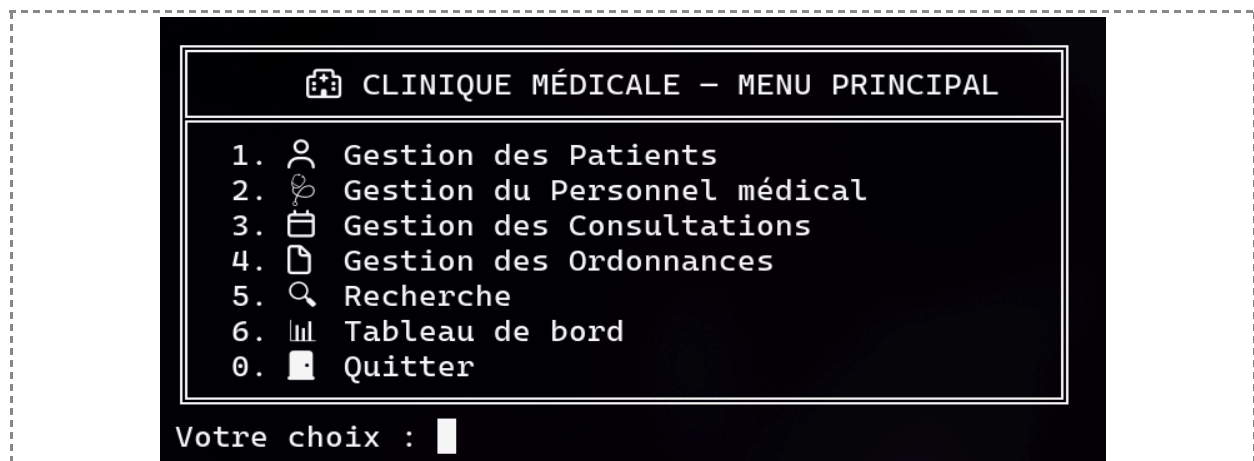


Figure 4.9 — Clinic dashboard output

5. Difficulties and Solutions

5.1 Designing a Coherent Class Hierarchy

One early challenge was deciding which classes should be abstract and which should be concrete. Making `PersonnelMedical` abstract in addition to `Personne` was not immediately obvious, but became the right choice once we confirmed that no object in the system would ever be a generic medical staff member without a specific role. This realization came from drawing the class diagram before writing any code.

5.2 Console Formatting Consistency

Keeping the console output readable and consistent across all menus required iterative adjustments. Fixed-width boxes using box-drawing characters would overflow when values were longer than anticipated. The solution was to compute the box width dynamically using `String.repeat()` and to use `printf` with explicit field widths for all rows.

We considered using specialized libraries to build an interactive **TUI** (Text User Interface), but we were concerned it might exceed the project's limitations, so we opted against it.

5.3 Exception Propagation

Understanding when to catch an exception locally versus when to propagate it to the calling menu took some iteration. The final approach was to catch exceptions in the menu layer (*`Main.java`*) where they can be displayed to the user, and to propagate them from the business layer (*`CliniqueMedicale.java`*) using throws declarations.

5.4 Doctor Availability Checking

Verifying doctor availability required iterating over all existing consultations and comparing date strings. This works correctly for the current scope but would not scale well to a large dataset.

6. Conclusion

This project successfully demonstrates the **five core concepts** of Object-Oriented Programming through a realistic and functional Java application. **Encapsulation** protects sensitive medical data, **inheritance** models the natural relationships between the actors of a clinic, **polymorphism** enables flexible and extensible code, **abstraction** enforces consistent contracts across the hierarchy, and **exception** handling produces clear and recoverable error behaviour.

6.1 Limitations

The current implementation has several known limitations that are acceptable within the scope of this exercise:

- Data is not persisted between sessions. Closing the application **loses** all records.
- Doctor availability is checked by exact date string match, which requires the user to enter dates **consistently**.
- The application has no authentication or access control mechanism.
- Search is limited to consultations; there is no full-text or multi-criteria search.
- Time is limited , we couldn't get an application level result , but it's decent.
- Art is sometimes inconsistent , we couldn't create a stable template for all possible values, so sometimes you will see some characters misplaced , but it won't affect the values themselves .

6.2 Possible Improvements

Several improvements could be made in a future version:

- Persist data to a file (JSON or CSV) or a relational database .
- Add a graphical user interface or a text user interface .
- Replace date strings with LocalDateTime objects for robust scheduling and conflict detection.